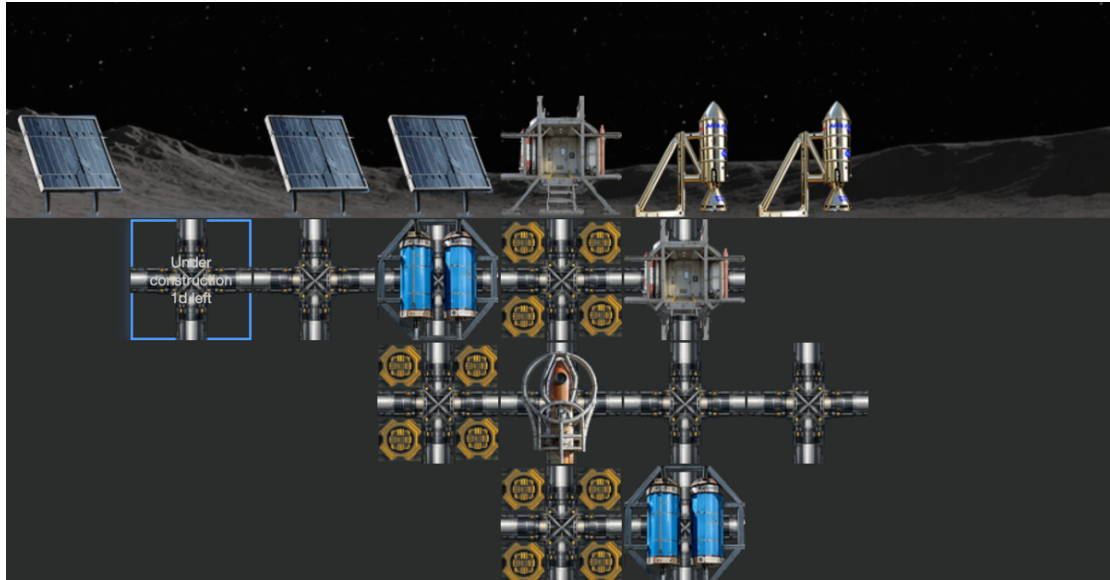


How Many Prompts Are Needed for a Turn-Based Strategy Game in JS?

My answer is: about 290 prompts for the code only. And 70 prompts for assets and deployment.



Base game view

The Project

I had the idea for this project for several months. It is a turn-based strategy game about space exploration, asteroid mining, and IPOs on space stocks. I made it for myself. I like strategy games.

Turn-based games are a useful case for this experiment because the technical model is relatively simple. You can apply changes to the game state when the turn ends. All events are caused by the player. There is no need for a constantly running server loop, and no database. In that sense, it is like a card game for one person. A version of solitaire, if you like.

Several weeks ago my 11-year-old son said something like this: "I prompted ChatGPT to create a new game for an iPad, like Mario, and it made something. But I don't understand what to do with the output. You are a programmer. Do you know what we should do with the output?"

Of course, I knew the answer: it does not work this way. I told him that it was impossible to create a real game with a single prompt.

Then I started thinking about the obvious next question: How many prompts does one need to create a game?

Scope

I started by formalizing the problem. How could I simplify the project without making the result meaningless? Which assumptions were reasonable?

I reduced the scope to this:

- A turn-based strategy game in JavaScript
- Estimated play time of about one hour per full game session
- Simple but understandable graphics
- No sounds
- HTML, CSS, and JavaScript only, because I did not want to use a game engine
- Static hosting with Nginx on the server side
- No database
- No registration
- No save/load

The result is not a general claim about all games. I did not try to build a real-time multiplayer shooter, a 3D world, or a mobile game with platform-specific code. I intentionally chose a game shape that could be finished with browser technologies and static hosting.

Development Setup

My setup:

- VS Code
- GitHub Copilot
- ChatGPT Plus in VS Code (Codex - OpenAI's coding agent)
- Krita with an AI Image Generation plugin using Flux 2 Klein for images
- The ChatGPT web version for general questions and architecture
- Chrome browser with the debug console

I started with GitHub Copilot and later switched mostly to ChatGPT. They are different tools, and for my style and for this particular game, ChatGPT worked better. I am not going to compare them in detail here. I do not have enough research material to make a serious statement about that.

Prompt Log

To answer the main question of this article, I recorded prompts and replies. I collected them in a JSON file with these fields:

You can find the log here:

<<https://pallasprospect.com/development-log.json>>

It contains "interaction_count": 287. I lost about the first 10 prompts, so the real number was around 290 or a little more for code only. Some records are from Copilot and some are from ChatGPT, but they are stored in the same format.

One practical problem was that it is difficult to extract these histories cleanly. Both tools keep local records, but I did not find a convenient way to export them in the structure I wanted. So I started by copy-pasting and formatting the history manually. Later I asked the AI itself to fill in the latest prompts and replies while preserving the JSON structure.

After more than 290 prompts for code, about 40 prompts for assets, and about 30 prompts for deployment, I can present the result in two forms:

- The game: `<https://pallasprospect.com/>`
- This article

Starting Conditions

I need to be clear about the starting conditions.

I already had the idea and the main details of the game in my mind before the development started. I did not need to ask AI what the game should be. I did not begin with "I want a game, create it for me."

I also worked before as a software developer in game development studios. I have about 20 years of programming experience in several languages and an MS in mathematics. I know how web services work. I know HTML, CSS, JavaScript, and how to debug them in a browser.

So this was not a non-programmer asking AI to create a complete product from nothing. And still, I needed more than 300 prompts and several weeks to create a game I like to play myself. The good part is that my son played it too.

What I Learned

This was a very interesting experience of AI-assisted development. I found both AI engines to be useful co-workers. Not magic, not autonomous developers, but useful co-workers.

Here are the main observations from the project:

LLMs Can Work Across Game Concepts and Code Concepts

It was acceptable to mix code-change commands with game-related descriptions. The LLM could understand when I was talking about an abstract game entity in a science-fiction setting and when I was talking about a code entity such as a class or an object. Sometimes the level of abstraction it handled was surprisingly high. For example, there was a rocket in the scene. I intended the player to disassemble the rocket later, so these cells would become available for building. But Copilot decided by itself to prevent building in those cells, arguing that "but there is a rocket there!".

The Work Often Moves From Code to Rules

At some point I noticed that half of the discussion was not about code. It was about game rules. In pre-AI game development, we often created special configuration layers for game designers. Designers could not change the code directly, so they changed configs. That was the safe and practical boundary between design and engineering. With LLM-assisted

development, this boundary changes. A designer can prompt changes directly into a local version of the code, test the idea, and later work with a software developer to include the solution in production code.

Small Examples Work Better Than Large Vague Requests

If many similar elements need to be changed, it is better to prepare one example first. Make the change for one object. Fix the process. Then ask the AI to apply the same pattern to the others. This worked much better than asking for a large transformation in one vague request. The AI is good at copying a local pattern once the pattern is visible.

AI Often Infers Reasonable UI Intentions

The example: the game had a table with several columns, and the last column had buttons. When I asked to add new columns to the table, the AI placed the new columns before the button column. That was exactly what I intended, but I did not explain it.

You Are the Tester

Your part of the work is not only to write prompts. Your part is to play the game after changes and find bugs. The AI makes bugs. In my experience, not more than a middle-level software developer would make on the same kind of tasks, but it still makes bugs. You need to test, debug, read code, and decide what is acceptable.

Tests Are Hard in Games

Testing is a problem in games. This is not mainly an AI problem. It is a game development problem. Test-driven development does not normally work well when the product does not have a strict specification and can change because the player does not feel what you want them to feel.

AI Generates Beautiful Spaghetti

AI can generate working code quickly. It can also generate beautiful spaghetti quickly. It is very willing to solve problems by adding many if-else branches. Object-oriented design, code organization, and architectural cleanup are still your responsibility. You need to check the code regularly, extract functionality, create classes, and prevent the project from becoming a pile of special cases.

The AI Understands More Textual Intent Than I Expected

Sometimes it understood instructions very precisely. For example, I asked:

In Market field in the row Raw Resources: at the right corner add a checkbox "Auto sell all".

It added the checkbox and implemented the correct "select all checkboxes" behavior. It also usually understood when I was asking only about text and when I was asking about text plus logic. I did not need to clarify this every time.

Visual Design Still Needs Human Judgment

The default visual design generated inside VS Code was bad. I improved it with the web version of ChatGPT, but even then I needed knowledge of CSS, HTML, and browser debugging. Without that knowledge, it would have been difficult to tune the visuals.

Sometimes It Loses the Thread

Two or three times during the project, the AI ignored a new prompt and answered the previous one again.

Project Memory Helps, But It Is Not Automatic

I asked the AI to record important information in README.md for me and in AGENTS.md for itself. This helped. But sometimes it forgot to update those files. I had to remember to ask for it.

The Practical Answer

So, how many prompts are needed to create a turn-based strategy game in JavaScript?

In my case:

- About 290 prompts for code
- About 40 prompts for assets
- About 30 prompts for deployment

The exact number is less important than the order of magnitude. It was not one prompt. It was not ten prompts. It was hundreds of small interactions over several weeks.

And those prompts only worked because there was already a clear idea, a limited scope, and a developer reading, testing, debugging, and making decisions.

Takeaway

Creating a game with AI is possible, but it is still real development work. AI changes the speed and shape of the work. It helps you move faster through code generation, UI changes, refactoring, asset descriptions, and implementation details. It can also help you think through rules and structure. But it does not remove the need for product judgment, programming knowledge, debugging, architecture, and playtesting.

It made this project easier and more fun. It also made the amount of hidden work more visible. A finished game is not the result of one perfect prompt. It is the result of many decisions, many corrections, and enough persistence to keep turning an idea into something playable.